

FORM-IC-04 — Preparación para Evaluación de Avance del 2do Corte

Portafolio individual de evidencias de aprendizaje

Universidad Técnica Estatal de Quevedo — Facultad de Ciencias de la Computación

Asignatura:	Aplicaciones Distribuidas (ISR-701)	Nivel:	7 ^{mo}
Docente:	Ing. Gleiston Cicerón Guerrero Ulloa	Equipo PFC:	ACC
Estudiante:	Cando Moreno Robinson Rodrigo		

1. Evidencias concretas de aprendizaje

Evidencia 1 — Instrumentación de métricas Prometheus en ticket-service

Se trabajó en la observabilidad del microservicio principal, agregando métricas específicas para el comportamiento de CockroachDB bajo carga concurrente: duración de queries, reintentos de transacción y conexiones activas del pool. Con esto se logró tener datos que después se pudieron cruzar con las pruebas de aislamiento serializable.

Reflexión: no estaba claro por qué un contador de reintentos es más útil que solo medir latencia promedio: la latencia puede verse normal y aun así estar ocurriendo contención seria si CockroachDB está reintentando transacciones por conflictos de serialización. Resolver esa duda llevó a revisar cómo funciona el modelo de aislamiento serializable de CockroachDB y en qué se diferencia de otros gestores de bases de datos que degradan a niveles más débiles bajo carga.

Evidencia 2 — Prueba de integración de aislamiento serializable con Testcontainers

Se incorporaron pruebas de integración contra una instancia real de CockroachDB, usando Testcontainers, con las que se verificó de forma empírica el reintento serializable ante escrituras concurrentes sobre el mismo ticket.

Análisis: lo más costoso no fue la prueba en sí, sino plantear un escenario de carrera real, con dos hilos escribiendo sobre el mismo registro al mismo tiempo, sin caer en falsos positivos por timing. Quedó claro que Testcontainers, al levantar un CockroachDB real en Docker, expone comportamientos como el reintento automático que un mock nunca mostraría.

Evidencia 3 — Baseline secuencial (pandas) vs. pipeline paralelo (Spark)

Se implementó el baseline en pandas y el protocolo experimental, con diez repeticiones e intervalo de confianza del 95%, usado para contrastar el tiempo de ejecución secuencial contra el pipeline distribuido de Spark sobre el dataset sintético de 600,000 filas.

Reflexión: afirmar que Spark es más rápido sin un protocolo estadístico sería un error. Con una sola corrida, la varianza del entorno, como el arranque de la JVM, el acceso a disco o la contención de CPU, puede simular una mejora que en realidad es ruido. Repetir diez veces y reportar un intervalo de confianza obligó a entender por qué la Ley de Amdahl predice un techo de aceleración que casi nunca se alcanza en la práctica, por el sobrecosto de shuffle, serialización y particionamiento desigual.

2. Contribución al PFC documentada con commits de GitHub

Commits registrados en el repositorio del proyecto bajo el usuario Robinson durante el segundo corte, ordenados de más reciente a más antiguo:

Fecha	Commit	Área
27 jul	fix(spark): corregir serialización de numpy.bool_ y agregar resultados reales del protocolo (D4.3, D4.2)	Spark / Experimentación
22 jul	feat(db-cluster): agregar esquemas, scripts de datos iniciales, herramientas de tolerancia a fallos y resultados de pruebas	BDD distribuida
22 jul	test(auth-service): agregar pruebas unitarias para flujos de autenticación y validación de zonas	Auth / Testing
20 jul	feat(spark): agregar baseline pandas y protocolo experimental JCR (r=10, IC95%)	Spark / Experimentación
20 jul	test(ticket-service): agregar pruebas de integración con Testcontainers contra CockroachDB real	BDD distribuida / Testing
20 jul	feat(ticket-service): instrumentar métricas Prometheus	Observabilidad
20 jul	test(ticket-service): actualizar pruebas para la nueva clave y el reintento serializable	BDD distribuida / Testing

El trabajo se concentró en tres frentes: persistencia distribuida, con esquemas, tolerancia a fallos y pruebas de integración serializable contra CockroachDB real; observabilidad, con métricas Prometheus; y el pipeline analítico, con el baseline en pandas y el protocolo experimental de Spark.

3. Valoración de mi nivel de logro por RA

Unidad 2 — Arquitecturas Distribuidas

Nivel: en proceso de consolidación (medianamente seguro).

Existe una comprensión clara de por qué una arquitectura de microservicios se prefiere sobre un monolito en un sistema como este, y de cómo la comunicación asíncrona por Kafka desacopla a los servicios entre sí. El diseño de APIs REST documentadas con OpenAPI y los conceptos de orquestación de contenedores, como Docker, Kubernetes y el API Gateway, se manejan todavía a un nivel más conceptual que aplicado.

Unidad 4 — Computación Paralela y Distribuida

Nivel: en proceso de consolidación (medianamente seguro).

Existe una comprensión sólida del paralelismo de datos y de por qué el rendimiento real de un pipeline distribuido se aleja del ideal que predice la Ley de Amdahl, apoyada en el contraste entre el baseline secuencial y el procesamiento en Spark. Los modelos de paralelismo a más bajo nivel, como MPI y OpenMP, y los paradigmas de computación en la nube como FaaS, serverless y edge computing, se manejan todavía a un nivel más teórico que aplicado.

4. Preguntas técnicas que más cuesta responder

1. Ley de Amdahl en la práctica: el protocolo experimental permite comparar el speedup real de Spark contra el baseline secuencial una vez ejecutado el experimento, pero estimar de antemano el techo teórico que predice la Ley de Amdahl requiere conocer con precisión qué porcentaje del pipeline corresponde a la fracción no paralelizable, es decir, al tiempo consumido por el shuffle, el I/O de disco y la recolección final de resultados. En un pipeline con varias transformaciones encadenadas, como el filtrado, el join por client_id, la ventana

de reincidencia y el clustering por texto, esa fracción no es evidente a simple vista. ¿Existe una forma sistemática de descomponer el tiempo de cada etapa del pipeline para calcular esa fracción antes de correr el experimento completo, en lugar de estimarla después a partir de los resultados obtenidos?

2. Control de concurrencia distribuida: CockroachDB resuelve los conflictos de escritura concurrente reintentando automáticamente las transacciones bajo aislamiento serializable, un comportamiento que se puede observar en las pruebas de integración contra el clúster real. Sin embargo, no es evidente cómo se relaciona ese mecanismo con un protocolo clásico de control de concurrencia como el bloqueo en dos fases distribuido, ni en qué se diferencia un reintento serializable de un interbloqueo real entre nodos. ¿Cómo distingue el sistema, en un escenario con varios nodos, entre un conflicto que se resuelve con un simple reintento y uno que requeriría detectar y romper un ciclo de espera entre transacciones ubicadas en nodos distintos del clúster?
3. Serverless y edge computing aplicados a un sistema ya desplegado como microservicios: pensando en un componente como el servicio de clasificación de tickets, que hoy corre como un microservicio con conexión persistente a una base de datos documental, no es evidente cómo se manejaría esa misma responsabilidad bajo un modelo de funciones como servicio, donde las instancias son efímeras y no mantienen estado ni conexiones abiertas entre invocaciones. Tampoco queda claro en qué escenarios concretos el procesamiento en el borde de la red aportaría una ventaja real frente a la arquitectura centralizada actual, pensada para un único proveedor de internet. ¿Qué criterios permitirían decidir si un componente de este tipo de sistema es un buen candidato para migrar a un modelo serverless o de edge computing, más allá del argumento general de escalabilidad?